



RISC-V S-mode Physical Memory Protection for Hypervisor

Editor - Bicheng Yang and José Martins

Version 0.2, 10/2025: This document is in development. Assume everything can change. See <http://riscv.org/spec-state> for details.

Table of Contents

Preamble	1
Copyright and license information.....	2
Contributors	3
1. Introduction.....	4
2. "Ssvsmp" Extension for VS-level Physical Memory Protection (vSPMP)	5
2.1. Extension Dependencies.....	5
2.2. Virtual SPMP (vSPMP).....	5
2.3. Access Method for vSPMP registers.....	6
3. "Ssvsmpen" Extension for Optimizing Context Switching of vSPMP Entries	7
3.1. Extension Dependencies	7
3.2. vSPMP Context Switch Optimization	7
4. "Sshsmpdeleg" Extension for Sharing Hardware Resources between SPMP and vSPMP	8
4.1. Extension Dependencies	8
4.2. Resource Sharing between SPMP and vSPMP	8
5. "Sshsmpen" Extension for Optimizing Context Switching of SPMP Hypervisor/Guest Entries	10
5.1. Extension Dependencies	10
5.2. Optimization for Context Switching of Hypervisor/Guest SPMP	10

Preamble



This document is in the [Development state](#)

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2026 by RISC-V International.

Contributors

The following individuals have contributed, directly or indirectly, to the proposed SPMP for Hypervisor specifications:

(Contributors are sorted alphabetically by last name.)

- José Martins, Editor <jose@osyx.tech>
- Bicheng Yang, Editor <bichengyang@sjtu.edu.cn>
- Haibo Chen
- Luís Cunha
- Ruud Derwig
- Dong Du
- Lupe Carlos IV
- Radim Krčmář
- Kajetan Nürnberger
- Sandro Pinto
- Manuel Rodriguez
- Thomas Roecker
- Peter Vrabel
- Harry Wagstaff
- Yubin Xia

Chapter 1. Introduction

The RISC-V S-mode Physical Memory Protection (SPMP) for Hypervisor integrates the SPMP and Hypervisor extensions by providing a two-stage SPMP. It introduces vSPMP, a first-stage SPMP, in control of VS-mode allowing a guest supervisor to specify memory protection policies within its guest physical address space. Guest VS/VU-mode accesses are further checked by the SPMP as defined by the "Supervisor-Level PMP Extensions, Version 1.0" .

Chapter 2. "Ssvsmp" Extension for VS-level Physical Memory Protection (vSPMP)

2.1. Extension Dependencies

1. The Ssvsmp is dependent on the latest versions of Supervisor-Level ISA specification (Version 1.13), and Shbare (Version 1.0) at the time of writing.
2. The Ssvsmp is dependent on the "Ssmp" Extension for S-level Physical Memory Protection (SPMP).

2.2. Virtual SPMP (vSPMP)

The Ssvsmp extension introduces a new memory protection layer called Virtual SPMP (vSPMP). The vSPMP behaves analogously to the baseline SPMP but is under the control of VS-mode and applies to memory accesses originating from VS and VU modes. The vSPMP is active only when $V = 1$. Conceptually, this extension establishes a two-stage SPMP hierarchy, preceding any checks by PMP or ePMP (see [Figure 1](#)). From a logical perspective, when both `vsatp.MODE` and `hgap.MODE` are set to Bare, memory accesses initiated by guest contexts (VS/VU) are subject to the following enforcement sequence:

1. vSPMP check: If the access is permitted by vSPMP, then
2. SPMP check: The access must also be permitted by SPMP.

If the vSPMP check, or both the vSPMP and SPMP checks, deny a VS/VU access, a standard SPMP page-fault directed at VS-mode is triggered using page fault exception codes 12 (instruction), 13 (load), or 15 (store/AMO). If only the SPMP denies a VS/VU access, a fault directed at HS-mode is raised using guest page fault exception codes as defined by Ssmp. Implementations may perform vSPMP and SPMP checks in parallel, provided the overall enforcement behavior matches the sequential logic described above.

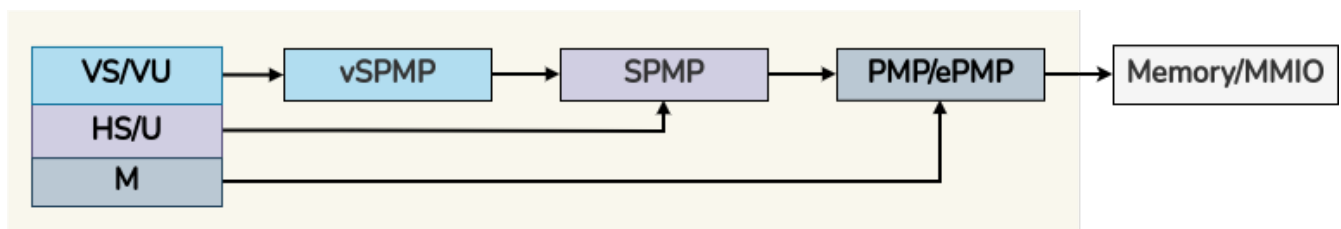


Figure 1. Dual-Stage SPMP

When `vsatp.MODE` is set to Bare but `hgap.MODE` is not, the vSPMP check is logically followed by G-stage address translation. Implementations may perform the vSPMP check and G-stage translation in parallel.

When `vsatp.MODE` is not set to Bare, but `hgap.MODE` is, the SPMP check logically occurs after the VS-stage address translation. In this scenario, the Guest Physical Address (GPA) produced by the VS-stage translation must be used as the input to the SPMP check. Consequently, parallel evaluation of VS-stage translation and SPMP enforcement is not possible unless speculative techniques are employed.

The vSPMP is configured through a set of virtual supervisor indirect-access registers, which are architectural replicas of the standard SPMP registers. These include `vspmpaddr0-vspmpaddr63`,

vspmpcfg0–vspmpcfg63. See [Section 2.3](#) for details on how these registers are accessed.

2.3. Access Method for vSPMP registers

Consistent with the behavior defined in Shbare and Sscsrind extensions, vSPMP registers can be accessed from M and HS-modes via the vsiselect/vsiregX and accesses to the original SPMP registers from VS-mode via siselect/siregX CSRs are transparently redirected to the corresponding virtual supervisor vSPMP registers.

Each vsiselect value selects one vSPMP entry. Indirect accesses via vsireg and vsireg2 access that entry's vspmpaddr and vspmpcfg, respectively. vsireg3, vsireg4, vsireg5, and vsireg6 are read-only 0.

vsiselect number	indirect CSR access of vsireg
vsiselect#0	vsireg → vspmpaddr[0], vsireg2 → vspmpcfg[0]
vsiselect#1	vsireg → vspmpaddr[1], vsireg2 → vspmpcfg[1]
...	...
vsiselect#63	vsireg → vspmpaddr[63], vsireg2 → vspmpcfg[63]

Both M and HS-mode accesses via vsireg2 CSRs, and VS-mode accesses via sireg2 CSRs to the vSPMP vspmpcfg registers, can set vspmpcfg[i].L to lock a vSPMP entry. When $V = 1$, and vspmpcfg[i].L is set, writes to that entry are ignored. Only M and HS-mode writes via vsireg2 can reset vspmpcfg[i].L.

When hstatus.VTVM is set and $V = 1$, any attempt to access vSPMP CSRs raises a virtual-instruction exception.

Indirect accesses to vSPMP CSRs are not ordered with respect to each other or with subsequent memory accesses. To enforce ordering for subsequent accesses whose effective privilege mode is VS or VU, software must execute memory memory-management fence instructions. When $V = 1$, a SFENCE.VMA instruction is required as defined in the base Ssmp extension. When $V = 0$, M or HS mode software must execute an HFENCE.VVMA instruction with $rs1=x0$ and $rs2=x0$, which synchronizes subsequent memory accesses with all preceding vSPMP CSR writes.

Chapter 3. "Ssvsmpen" Extension for Optimizing Context Switching of vSPMP Entries

3.1. Extension Dependencies

1. The Ssvsmpen is dependent on Svsmp.

3.2. vSPMP Context Switch Optimization

The Ssvsmpen is an optional extension that provides a mechanism to enhance vSPMP context switches. It replicates the functionality provided by Svsmp for optimizing SPMP context switches for the vSPMP. For this, it introduces new virtual supervisor CSRs that control the activation of vSPMP entries:

- For RV64 architectures, it introduces a 64-bit WARL virtual CSR, vspmpen.
- For RV32 architectures, it introduces an additional 32-bit WARL CSR, vspmpenh, which serves as an alias for the upper 32 bits of the vspmpen register.

When `hstatus.VTVM` is set and when $V = 1$, any attempt to access vspmpen raises a virtual-instruction exception.

This extension is mandatory if Svsmp is implemented.

Chapter 4. "Sshsmpdeleg" Extension for Sharing Hardware Resources between SPMP and vSPMP

4.1. Extension Dependencies

- 1. The Sshsmpdeleg is dependent on Ssvsmp and Smpmpdeleg.

4.2. Resource Sharing between SPMP and vSPMP

With this extension the hypervisor can delegate entries from SPMP to vSPMP. For this, it introduces a new WARL HSXLEN-bit HS-mode CSR named hspmpdeleg, whose layout is detailed in [Figure 2](#). This CSR with the same functionality of mpmmpdeleg, but for delegating SPMP entries to vSPMP. Any PMP entry with an index greater than or equal to mpmmpdeleg.pmpnum + hspmpdeleg.pmpnum is delegated to the vSPMP. For example, if the number of implemented PMP entries is 48, mpmmpdeleg.pmpnum is set to 8, and hspmpdeleg.pmpnum is set to 16, PMP entries 24 to 47 are delegated to the vSPMP.

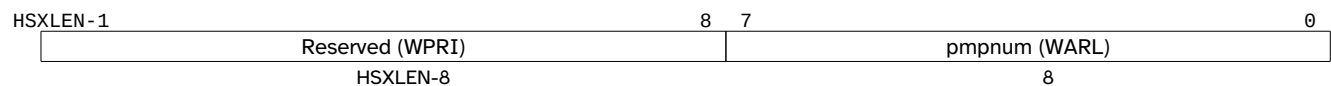


Figure 2. hspmpdeleg CSR format.

If either mpmmpdeleg.pmpnum or hspmpdeleg.pmpnum is written with a value such that mpmmpdeleg.pmpnum + hspmpdeleg.pmpnum is equal to or greater than the number of implemented PMP entries, hspmpdeleg.pmpnum will read back as the number of remaining SPMP entries. For example, if the number of implemented PMP entries is 32, mpmmpdeleg.pmpnum is set to 16, and then hspmpdeleg.pmpnum is written a value of 32, the field will read back as 16. Or if initially, mpmmpdeleg.pmpnum is set to 8, hspmpdeleg.pmpnum to 20 and then mpmmpdeleg.pmpnum is written 16, hspmpdeleg.pmpnum is updated to 16. In the extreme, if mpmmpdeleg.pmpnum is set to 32, hspmpdeleg.pmpnum must be updated to 0.

hspmpdeleg.pmpnum must be set to a value larger than the index of any locked SPMP entry. For example, if SPMP[7] is locked, hspmpdeleg.pmpnum must be no less than 8. hspmpdeleg.pmpnum is allowed to be updated to a value less than or equal to the index of any locked SPMP entry only due to a write to mpmmpdeleg.pmpnum.

hspmpdeleg.pmpnum can be hardwired to allow an implementation to fix the SPMP/vSPMP split only if mpmmpdeleg.pmpnum is also hardwired.

This extension allows for the number of implemented PMP entries to be greater than 64 and up to a maximum of 192. If the number of implemented entries is greater than 64, unless hardwired, mpmmpdeleg.pmpnum reset value is 64, and hspmpdeleg.pmpnum is set to the value of the remaining entries available to the SPMP. For example, if the total number of entries is 96, hspmpdeleg.pmpnum reset value is 32.



The limit of 192 for implemented PMP entries stems from the fact that this is the maximum number of addressable entries: 64 addressable for the vSPMP, 64 for the SPMP, plus 64 for PMP. If more PMP entries were implemented, software would have no way to address and access such entries.



The vSPMP can be effectively disabled by setting hspmpdeleg.pmpnum to the maximum possible value. If, for example, 96 entries are implemented, and mpmmpdeleg.pmpnum is set to 32, the vSPMP is disabled by setting hspmpdeleg.pmpnum to 64. In the extreme

case, when all 192 entries are implemented, and `mpmpdeleg.pmpnum` is set to zero, HS-mode can disable vSPMP by setting `hspmpdeleg.pmpnum` to 192.

Addressing:

vSPMP entries will be supported contiguously and begin with the lowest vSPMP indirect CSR number. For instance, given an implementation with a total of 48 PMP entries, if `mpmpdeleg.pmpnum` is set to 16, and `hspmpdeleg.pmpnum` to 16, PMP entry[32] to PMP entry[47] would map to vSPMP[0] to vSPMP[15]. A read of an out-of-index vSPMP entry (in the previous example, starting with vSPMP[16]) will return 0, and a write to such a vSPMP entry will be ignored.

Indirect accesses to vSPMP CSRs are not ordered with respect to each other or with subsequent memory accesses. To enforce ordering for subsequent accesses whose effective privilege mode is VS or VU, upon access to vSPMP CSRs, M- and HS-mode software must execute an HFENCE.VVMA instruction with `rs1=x0` and `rs2=x0`.



If `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum` is configured such that more than 64 entries are delegated to vSPMP, only the lowest 64 delegated entries are addressable through the vSPMP CSRs. Any additional delegated entries are architecturally inaccessible and have no effect on memory protection. For example, if the number of implemented PMP entries is 128, `mpmpdeleg.pmpnum` is set to 16, and `hspmpdeleg.pmpnum` is set to 16, PMP entry[32] to PMP entry[95] would map to vSPMP[0] to vSPMP[63], and PMP entry[96] to PMP entry[127] fall outside the vSPMP addressable range and therefore have no effect on memory protection.

This extension is mandatory if `Ssvsmp` is implemented.

Chapter 5. "Sshspmpen" Extension for Optimizing Context Switching of SPMP Hypervisor/Guest Entries

5.1. Extension Dependencies

1. The Sshspmpen is dependent on Ssmp.

5.2. Optimization for Context Switching of Hypervisor/Guest SPMP

The Sshspmpen introduces additional CSRs:

- For RV64 architectures, it introduces a 64-bit WARL CSR, hspmpen.
- For RV32 architectures, it introduces an additional 32-bit WARL CSR, hspmpenh, which serves as an alias for the upper 32 bits of the hspmpen register.

When $V = 1$, these registers replace the function of spmpen. Specifically, when $V = 1$, an SPMP entry is considered active only if both its `spmpcfg[i].A` is set and the corresponding bit in hspmpen is set. When $V = 0$, the hspmpen register has no effect, and the baseline SPMP behavior applies as defined by spmpen alone.

When an entry i is locked, as indicated by `spmpcfg[i].L == 1`, bit i in hspmpen becomes read-only.



The rationale for providing a separate hspmpen register, rather than reusing spmpen, is that HS/U and VS/VU contexts conceptually occupy distinct address spaces, not merely different permission domains. This mirrors the baseline Hypervisor extension for page-based translation, where HS/U accesses use the page tables pointed to by satp, while VS/VU accesses are translated through a separate set of page tables defined by hgatp. Similarly, spmpen governs SPMP entries for the hypervisor's own address space, while hspmpen controls the guest address space enforced by SPMP. This separation allows the hypervisor to manage its own and guest memory protection configurations independently.